

Stack

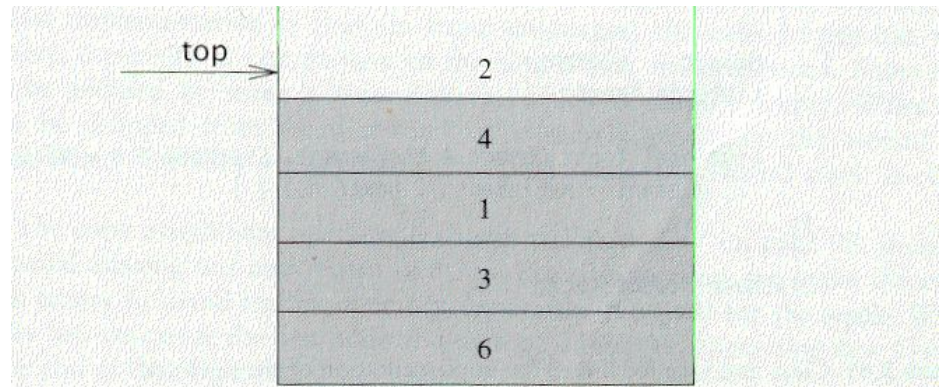
Stack Overview

- Stack ADT
- Basic operations of stack
 - Pushing, popping etc..
- Implementations of stacks using
 - array
 - linked list



The Stack ADT

- A stack is a list with the restriction
 - that insertions and deletions can only be performed at the *top* of the list



- The other end is called bottom
- Fundamental operations:
 - Push: Equivalent to an insert
 - Pop: Deletes the most recently inserted element
 - Top: Examines the most recently inserted element

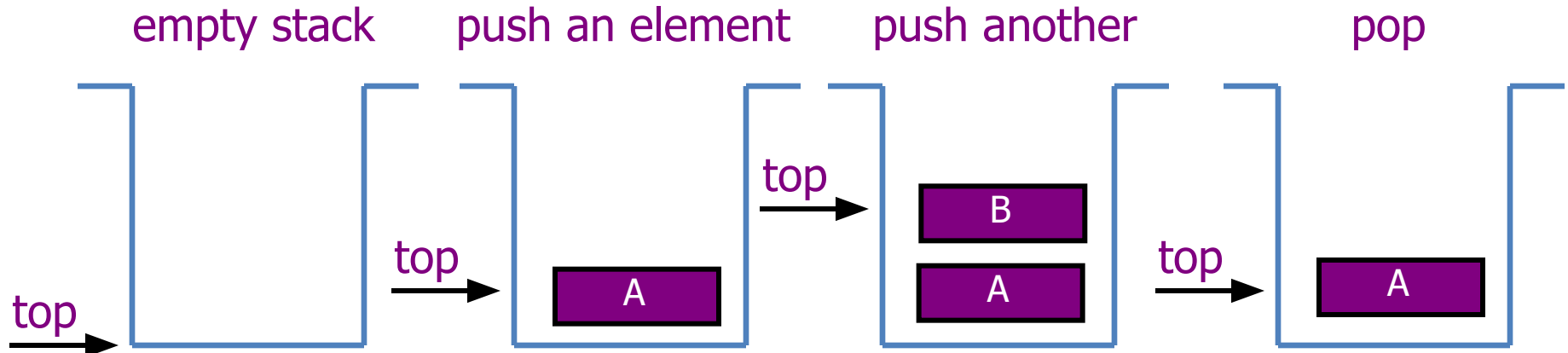
Stack ADT

- Stacks are less flexible
 - ✓ but are more efficient and easy to implement
- Stacks are known as **LIFO** (Last In, First Out) lists.
 - The last element inserted will be the first to be retrieved

Operations

Primary operations: **Push** and **Pop**

- **Push** - Add an element to the top of the stack
- **Pop** - Returns and removes the element at the top of the stack
- **Top/ Peek** - Returns the element at the top of the stack (no removal)
- **IsEmpty** - Check whether stack is empty or not, returns boolean value
- **Size** – Returns number of elements present in the stack



Implementation of Stacks

- Any list implementation could be used to implement a stack
 - Arrays (**static**: the size of stack is given initially)
 - Linked lists (**dynamic**: never become full)
- We will explore implementations based on array and linked list
- Let's see how to use an **array** to implement a stack first

Array Implementation

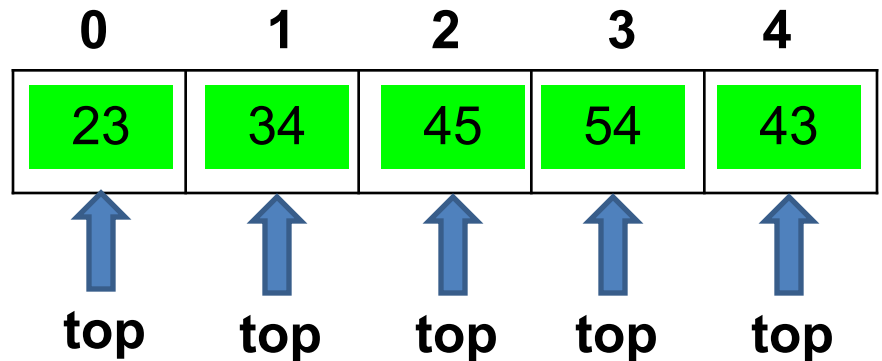
- Need to declare an array size ahead of time
- Associated with each stack is `topOfStack`
 - for an empty stack, set `topOfStack` to -1
- Push
 - (1) Increment `topOfStack` by 1.
 - (2) Set `Stack[topOfStack] = X`
- Pop
 - (1) Set return value to `Stack[topOfStack]`
 - (2) Decrement `topOfStack` by 1
- These operations are performed in constant time $O(1)$

Array Implementation

Stack of five elements created

top = -1

1. Pop (underflow)
2. Push 23
3. Push 34
4. Top (returns 34)
5. Push 45
6. Push 56
7. Pop (returns 56)
8. Push 54
9. Push 43
10. Push 32 ($\text{top}+1 > n-1$, overflow)
11. Pop (returns 43)



Applications of Stack

- Recursion
- Back-tracking
- As auxiliary data structures

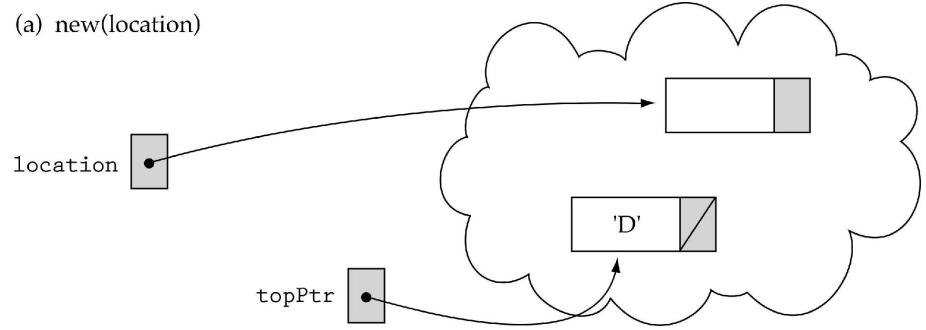
Specific examples-

- Page-visited history in a Web browser
- Undo sequence in a text editor

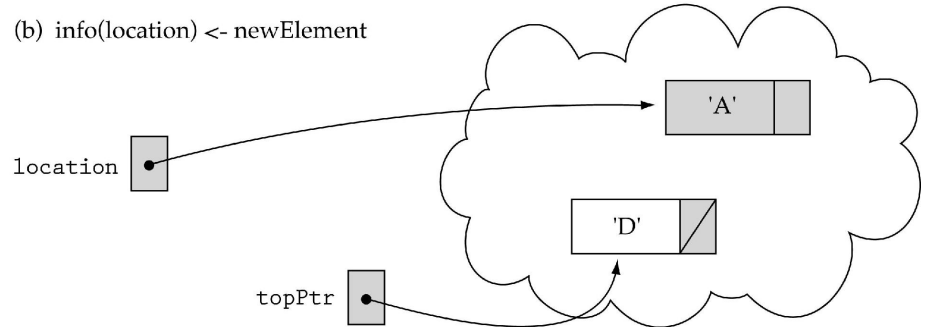
Linked list representation

- Push

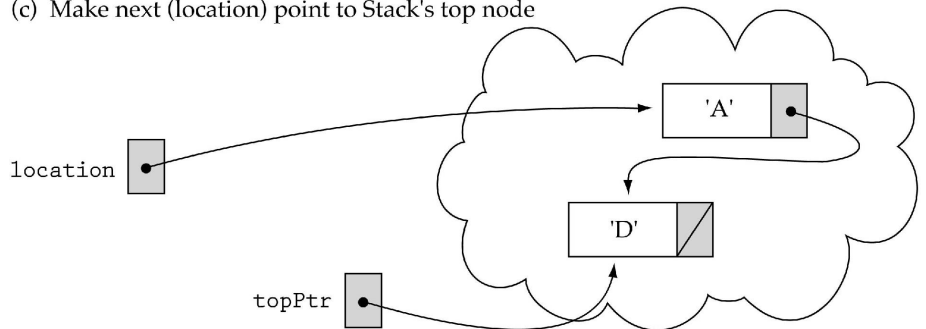
(a) `new(location)`



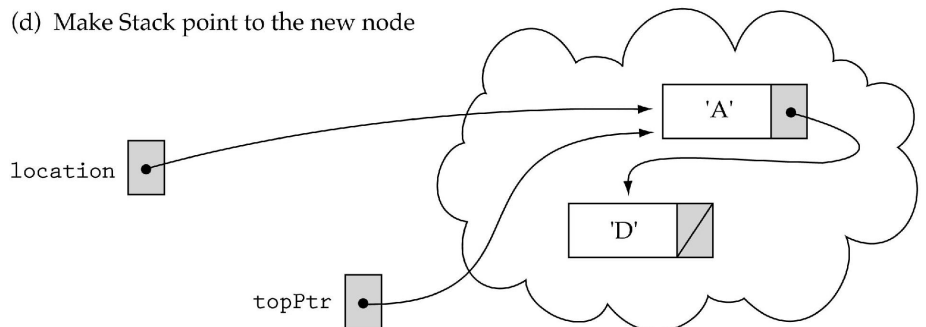
(b) `info(location) <- newElement`



(c) Make next (`location`) point to Stack's top node



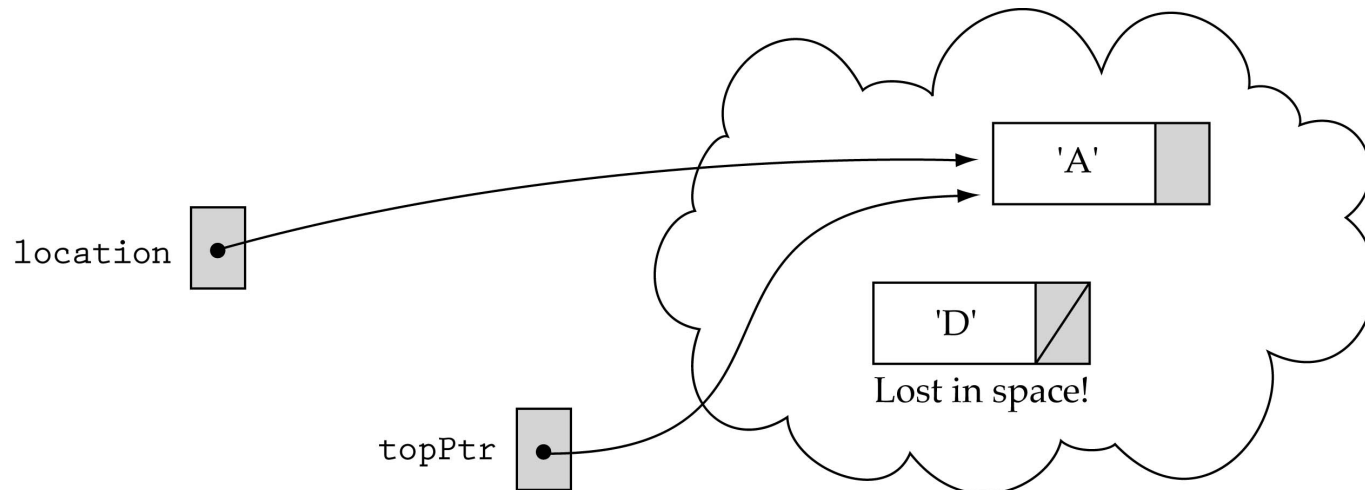
(d) Make Stack point to the new node



Linked list representation

Pushing in a non-empty stack

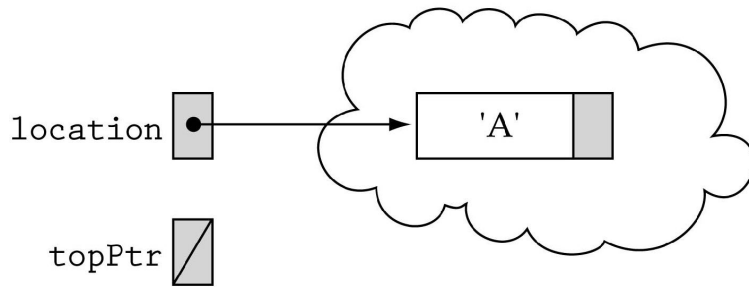
- The order of changing the pointers is very important !!



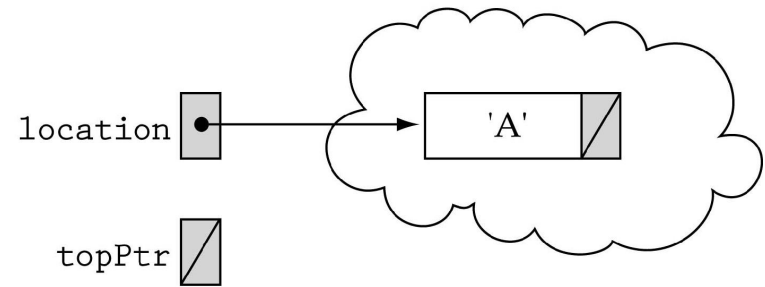
Linked list representation

Pushing on an empty stack

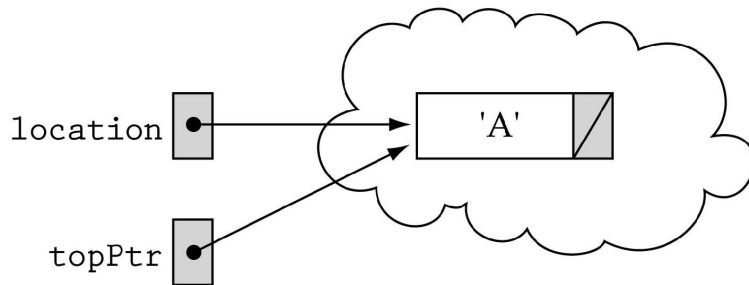
(a)



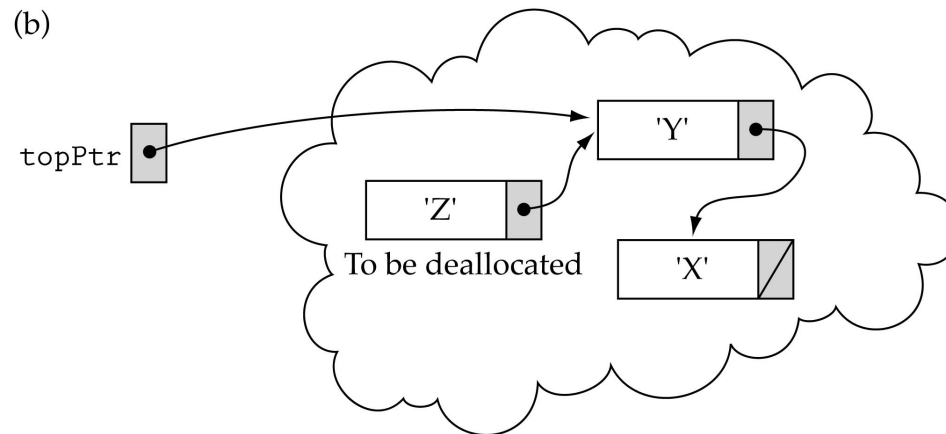
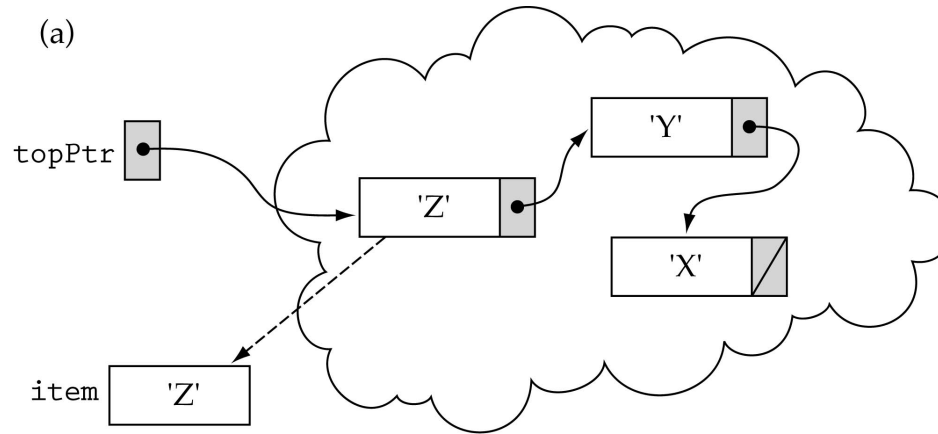
(b)



(c)



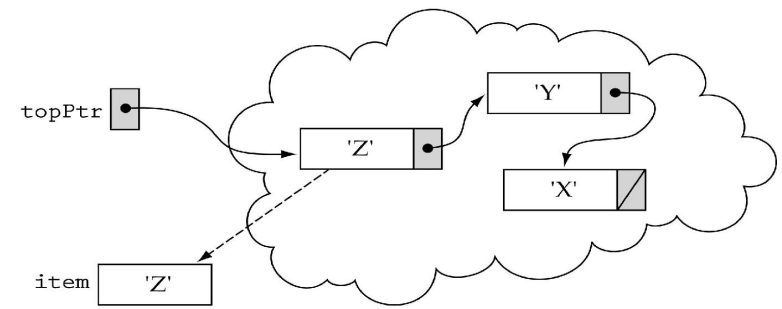
Popping the top element



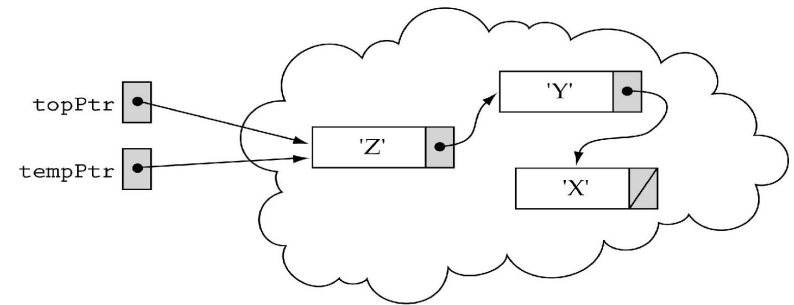
Popping the top element

Need to use a temporary pointer

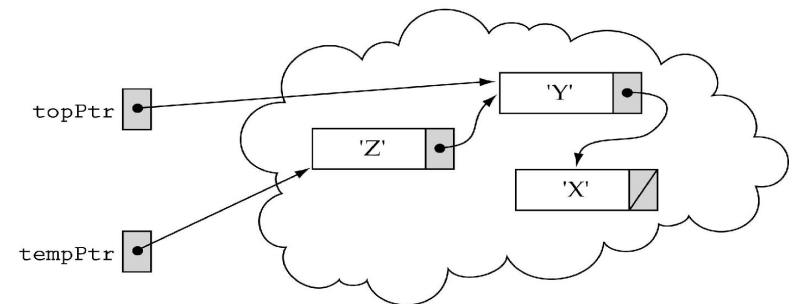
(a)



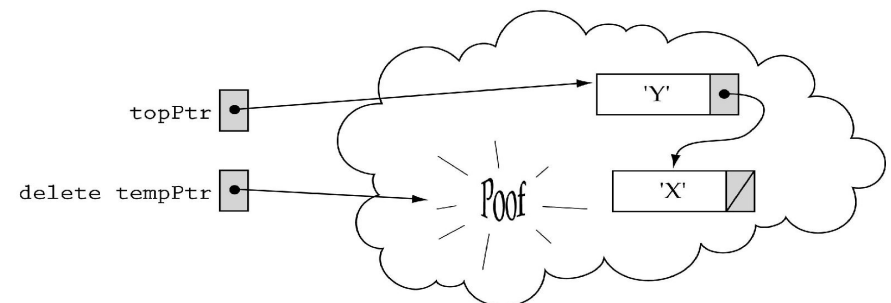
(b)



(c)

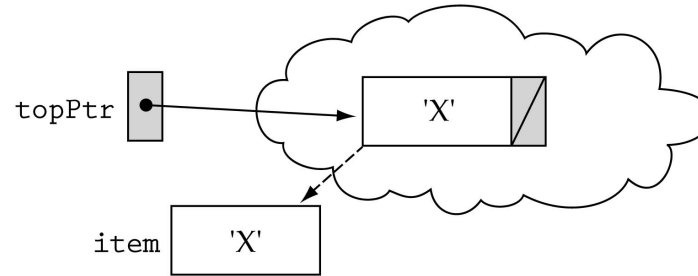


(d)

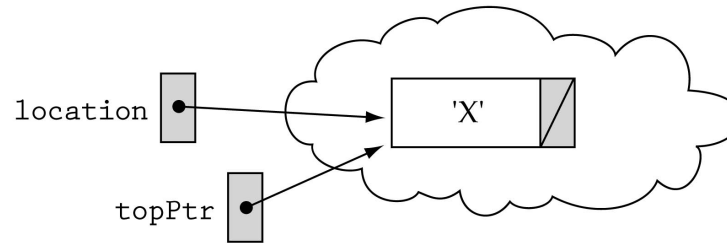


Popping the last element on the stack

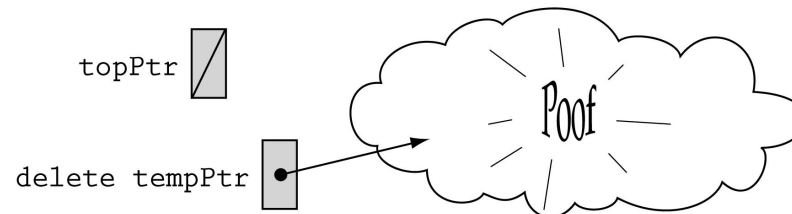
(a)



(b)



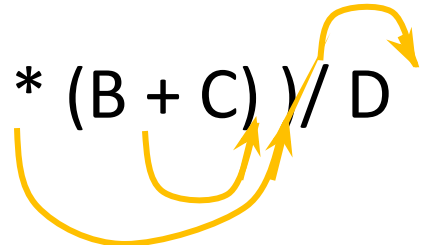
(c)



Arithmetic Expressions

Infix notation : $(A * (B + C)) / D$

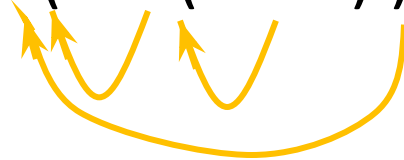
Postfix or reverse Polish notation : $(A * (B + C)) / D$



A B C + * D /

Prefix or Polish notation :

$(A * (B + C)) / D$



/ * A + B C D

Building an Arithmetic Expression Evaluator

Infix and Postfix Expressions:

*Assume 1-digit integer operands
and the binary operators + - * / only*

Infix Expression Properties:

- * Usual precedence and associativity of operators
- * Parentheses used to subvert precedence

Postfix Expression Properties:

- * Both operands of binary operators precede operator
- * Parentheses no longer needed

<u>Infix Expression</u>	<u>Equivalent Postfix</u>
<u>Expression</u>	
$3 * 4 + 5$	$34 * 5 +$
$3 * (4 + 5) / 2$	$345 + * 2 /$
$(3 + 4) / (5 - 2)$	$34 + 52 - /$
$7 - (2 * 3 + 5) * (8 - 4 / 2)$	$723 * 5 + 842 / - * -$
$3 - 2 + 1$	$32 - 1 +$

Building an Arithmetic Expression Evaluator

*Assume 1-digit integer operands, the binary operators + - * / only, and the string to be evaluated is properly formed*

Postfix Expression String Processing

Rules for processing the postfix string:

Starting from the left hand end, inspect each character of the string

1. if it's an operand – push it on the stack
2. if it's an operator – remove the top 2 operands from the stack, perform the indicated operation, and push the result on the stack

An Example: $3 * (4 + 5) / 2$ $345+*2/$ 13

<u>Remaining Postfix String</u>	<u>int Stack (top)</u>	<u>Rule Used</u>
---------------------------------	------------------------	------------------

345+*2/		
---------	--	--

45+*2/	3	
--------	---	--

5+*2/	3 4	
-------	-----	--

1+*2/	3 4 5	
-------	-------	--

12/	3 9	
-----	-----	--

2/	27	
----	----	--

/ 2	27 2	
-----	------	--

null	13	2
------	----	---

value of expression at top of stack

Expression	Action	Stack
5 6 * 10 - ↑	Push 5	5
5 6 * 10 - ↑	Push 6	6 5
5 6 * 10 - ↑	Pop 6 and 5 Evaluate 5 * 6 Push 30	30
5 6 * 10 - ↑	Push 10	10 30
5 6 * 10 - ↑	Pop 10 and 30 Evaluate 30 - 10 Push 20	20
5 6 * 10 - ↑	Pop 20 Stack is empty Result is 20	

Building an Arithmetic Expression Evaluator

*Assume 1-digit integer operands, the binary operators + - * / only, and the string to be converted is properly formed*

Infix to Postfix Conversion

Rules for converting the infix string:

Starting from the left hand end, inspect each character of the string

1. if it's an operand – append it to the postfix string
2. if it's a '(' – push it on the stack
3. if it's an operator – if the stack is empty, push it on the stack else pop operators of greater or equal precedence and append them to the postfix string, stopping when a '(' is reached, an operator of lower precedence is reached, or the stack is empty; then push the operator on the stack
4. if it's a ')' – pop operators off the stack, appending them to the postfix string, until a '(' is encountered and pop the '(' off the stack
5. when the end of the infix string is reached – pop any remaining operators off the stack and append them to the postfix string

Infix to Postfix Conversion (continued)

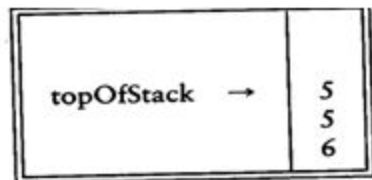
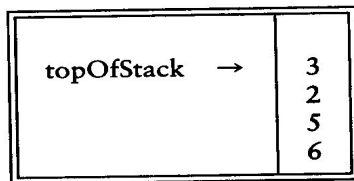
An Example: $7 - (2 * 3 + 5) * (8 - 4 / 2)$ $\square$ $723 * 5 + 842 / - * -$

<u>Remaining Infix String</u>	<u>char Stack</u>	<u>Postfix String</u>	<u>Rule Used</u>
$7 - (2 * 3 + 5) * (8 - 4 / 2)$	empty	null	
$- (2 * 3 + 5) * (8 - 4 / 2)$	empty	7	1
$(2 * 3 + 5) * (8 - 4 / 2)$	-	7	3
$2 * 3 + 5) * (8 - 4 / 2)$	- (	7	2
$* 3 + 5) * (8 - 4 / 2)$	- (	72	1
$3 + 5) * (8 - 4 / 2)$	- (*	72	3
$+ 5) * (8 - 4 / 2)$	- (*	723	3
$5) * (8 - 4 / 2)$	- (+	723*	3
$) * (8 - 4 / 2)$	- (+	723*5	1
$* (8 - 4 / 2)$	-	723*5+	4
$(8 - 4 / 2)$	-*	723*5+	3
$8 - 4 / 2)$	-* (	723*5+	2
$- 4 / 2)$	-* (	723*5+8	1
$4 / 2)$	-* (-	723*5+8	3
$/ 2)$	-* (-	723*5+84	1
$2)$	-* (- /	723*5+84	3
$)$	-* (- /	723*5+842	1
null	empty	$723 * 5 + 842 / - * -$	4&5

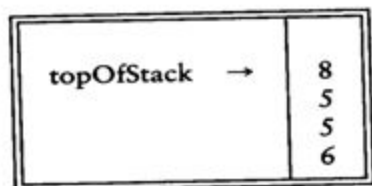
Conversion of $x1 + 2.5 * count / 3$

Next Token	Action	Effect on operatorStack	Effect on postfix
x1	Append x1 to postfix.		x1
+	The stack is empty Push + onto the stack		x1
2.5	Append 2.5 to postfix		x1 2.5
*	precedence(*) > precedence(+), Push * onto the stack		x1 2.5
count	Append count to postfix		x1 2.5 count
/	precedence(/) equals precedence(*) Pop * off of stack and append to postfix		x1 2.5 count *
/	precedence(/) > precedence(+), Push / onto the stack		x1 2.5 count *
3	Append 3 to postfix		x1 2.5 count * 3
End of input	Stack is not empty, Pop / off the stack and append to postfix		x1 2.5 count * 3 /
End of input	Stack is not empty, Pop + off the stack and append to postfix		x1 2.5 count * 3 / +

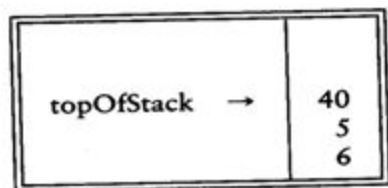
- Example: evaluate $6\ 5\ 2\ 3\ +\ 8\ *\ +\ 3\ +\ *$
- The time to evaluate a postfix expression is $O(n)$
 - processing each element in the input consists of stack operations and thus takes constant time



Next 8 is pushed.

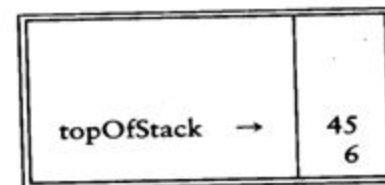


Now a '*' is seen, so 8 and 5 are popped and $5 * 8 = 40$ is pushed.

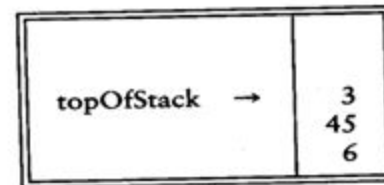


Next a '+' is seen, so 40 and 5 are popped and $5 + 40 = 45$ is pushed

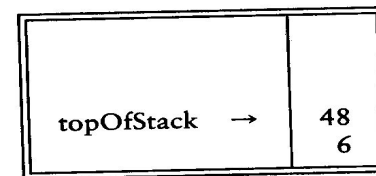
Next a '+' is seen, so 40 and 5 are popped and $5 + 40 = 45$ is pushed



Now, 3 is pushed.



Next '+' pops 3 and 45 and pushes $45 + 3 = 48$.



and 48 and 6 are popped; the result, $6 * 4$

